Documentație Tema 1

Aplicație pentru Administrarea unei Grădini Botanice

Student:

Coclici Delia Maria

Grupa:

30235

25 martie 2026

Cuprins

1 Enunț **2**

2 Instrumente utilizate **2**

3 Justificare limbaj de programare utilizat **2**

4 Descriere diagrame UML **3**

4.1 Faza de analiză . . . 3

4.1.1 Diagrama cazurilor de utilizare . . . 3

4.1.2 Diagrame de activități . . . 4

4.2 Faza de proiectare . . . 6

4.2.1 Diagrama de pachete . . . 6

4.2.2 Diagrama de clase . . . 6

4.2.3 Diagrama entitate-relație . . . 7

5 Descriere interfață grafică utilizator **9**

5.1 Tab-ul Plante . . . 9

5.2 Tab-ul Exemplare . . . 10

5.3 Tab-ul Listă . . . 11

6 Concluzii **11**

6.1 Obiective realizate . . . 11

6.2 Aspecte tehnice . . . 12

6.3 Posibile îmbunătățiri . . . 12

6.4 Experiență dobândită . . . 12

1 Enunț

Dezvoltați o aplicație software care poate fi utilizată într-o grădină botanică pentru administrarea plantelor. Softul va permite:

- Adăugarea, ștergerea și actualizarea informațiilor despre plantele din grădina botanică (denumire, tip, specie, plante carnivore, etc.);
- Vizualizarea listei tuturor plantelor din grădina botanică sortată după tip și specie (vizualizarea include și redarea unor imagini cu toate plantele din grădina botanică);
- Filtrarea listei plantelor după următoarele criterii: tip, specie sau plante carnivore;
- Adăugarea, ștergerea și actualizarea informațiilor despre exemplarele speciilor din grădina botanică;
- Vizualizarea listei exemplarelor (imagini, zona din grădina botanică) din grădina botanică;
- Căutarea tuturor exemplarelor dintr-o anumită specie.

2 Instrumente utilizate

Pentru dezvoltarea acestei aplicații au fost utilizate următoarele instrumente și tehnologii:

Instrument	**Scop**
Java	Logica aplicației și interfața grafică
Maven	Sistem de management al proiectului
IntelliJ IDEA	Mediu de dezvoltare (IDE)
MySQL	Sistem de gestiune a bazei de date
Draw.io	Crearea diagramelor UML

Tabela 1: Instrumente utilizate în dezvoltarea aplicației

3 Justificare limbaj de programare utilizat

Limbajul Java a fost ales pentru dezvoltarea acestei aplicații din următoarele motive:

1. **Portabilitate ridicată** – aplicația se poate rula pe diverse sisteme de operare fără modificări majore în cod, datorită mașinii virtuale Java (JVM).

2. **Suport puternic pentru baze de date** – Java oferă JDBC (Java Database Connectivity) care simplifică integrarea cu baze de date relaționale precum MySQL.
3. **Gestiune eficientă a fișierelor** – bibliotecile standard Java permit manipularea ușoară a fișierelor și resurselor.
4. **Paradigma orientată pe obiecte** – permite o structură clară și modulară a codului, facilitând mentenanța și extinderea aplicației.

4 Descriere diagrame UML

4.1 Faza de analiză

Faza de analiză cuprinde diagramele care descriu cerințele funcționale ale sistemului și fluxurile de activități din perspectiva utilizatorului.

4.1.1 Diagrama cazurilor de utilizare

Diagrama cazurilor de utilizare descrie funcționalitățile pe care le poate accesa utilizatorul aplicației:

Gestionare plante

- Adăugare plantă
- Editare plantă
- Ștergere plantă

Gestionare exemplare specie

- Adăugare exemplar specie
- Editare exemplar specie
- Ștergere exemplar specie
- Căutare exemplar dintr-o anumită specie

Vizualizare liste

- Listare plante
- Listare exemplare specie
- Filtrare plante după tip și specie

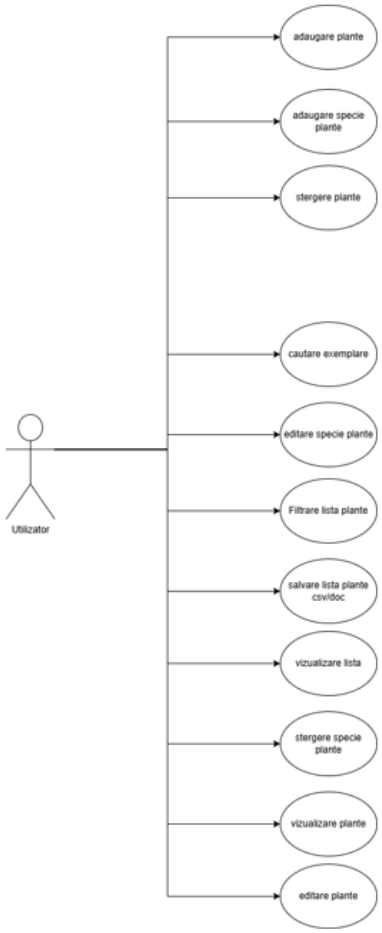

Figura 1: Diagrama cazurilor de utilizare

4.1.2 Diagrame de activități

Diagramele de activități ilustrează interacțiunea dintre componentele aplicației în procesul de gestionare a datelor:

1. Se adaugă datele pe care utilizatorul dorește să le introducă
2. `DesktopGui` trimite datele către `GradinaBotanicaPresenter`
3. `GradinaBotanicaPresenter` validează și trimite mai departe datele către `PlantaRepo` sau `ExemplarRepo`
4. Repository-urile salvează datele în baza de date

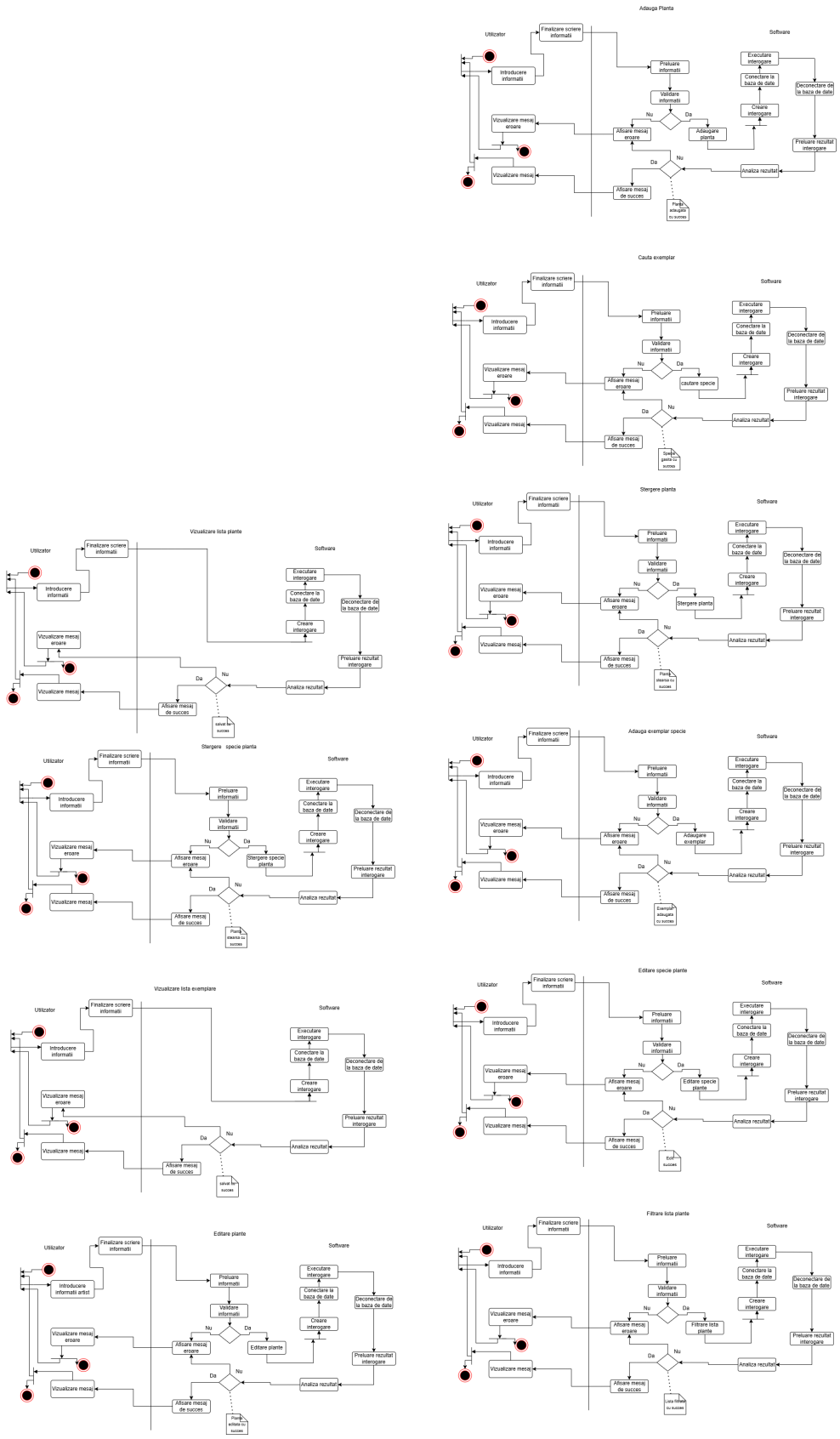

Figura 2: Diagramele de activități

4.2 Faza de proiectare

Faza de proiectare cuprinde diagramele care descriu arhitectura tehnică a sistemului, structura claselor și modelul bazei de date.

4.2.1 Diagrama de pachete

Diagrama de pachete descrie arhitectura aplicației bazată pe pattern-ul **Model-View-Presenter (MVP)**:

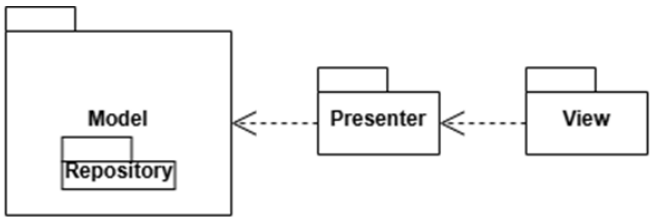

Figura 3: Diagrama de pachete – Arhitectura MVP

Pachet	Responsabilitate
Model	Conține clasele care reprezintă entitățile din domeniu (`Planta`, `Exemplar`)
View	Conține interfața grafică a aplicației
Presenter	Conține logica de prezentare și coordonează comunicarea dintre Model și View
Repository	Conține clasele pentru accesul la baza de date

Tabela 2: Descrierea pachetelor aplicației

4.2.2 Diagrama de clase

Diagrama de clase prezintă structura aplicației și relațiile dintre clase, organizate conform arhitecturii MVP (Model-View-Presenter):

Model

- **Planta** – reprezintă o plantă, având atributele: `id`, `denumire`, `tip`, `specie`, `planta_carnivora`, `imagine`.
- **Exemplar** – reprezintă un exemplar de specie, având atribute precum: `id`, `zona`, `imagine`, `plantaId`, `denumire`.

Repository

- **ExemplarRepository** – gestionează operațiile CRUD pentru exemplarele de specie.
- **PlantaRepository** – gestionează operațiile CRUD pentru plante.

Presenter

- **GradinaBotanicaPresenter** – coordonează interacțiunea dintre `DesktopAplic`, `PlantaRepository` și `ExemplarRepository`.

View

- **DesktopAplic** – interfața grafică principală a aplicației.

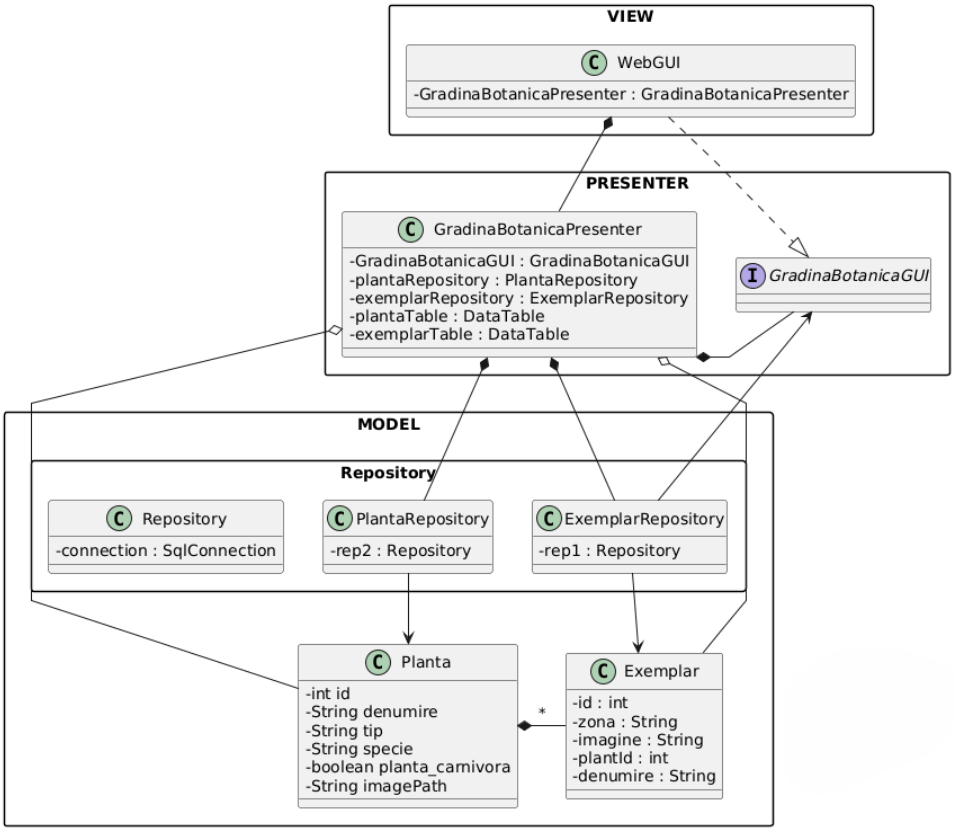

Figura 4: Diagrama de clase

4.2.3 Diagrama entitate-relație

Diagrama entitate-relație prezintă structura bazei de date și relațiile dintre tabele. Aplicația utilizează două tabele principale: `Planta` și `Exemplar`, cu o relație de tip unul-la-multe între ele.

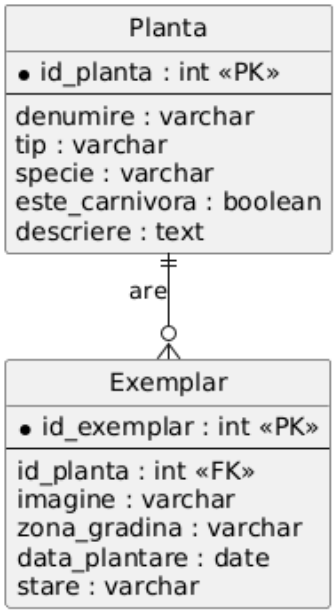

Figura 5: Diagrama entitate-relație

Câmp	**Tip**	**Constrângeri**	**Descriere**
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificator unic
denumire	VARCHAR(255)	NOT NULL	Numele plantei
tip	VARCHAR(100)		Tipul plantei
specie	VARCHAR(100)		Specia plantei
planta_carnivora	BOOLEAN		Indicator plantă car
imagine	VARCHAR(500)		Calea către imagine

Tabela 3: Structura tabelei Planta

Câmp	**Tip**	**Constrângeri**	**Descriere**
id	INT	PRIMARY KEY, AUTO_INCREMENT	Identificator unic
zona	VARCHAR(100)		Zona din grădină
imagine	VARCHAR(500)		Calea către imagine
planta_id	INT	FOREIGN KEY	Referință la plantă
denumire	VARCHAR(255)		Denumirea exemplarului

Tabela 4: Structura tabelei Exemplar

5 Descriere interfață grafică utilizator

Interfața grafică a aplicației este organizată în trei tab-uri principale: **Plante**, **Exemplare** și **Listă**.

5.1 Tab-ul Plante

Acest tab permite gestionarea completă a plantelor din grădina botanică:

Secțiunea „Detalii plantă" Conține câmpurile pentru introducerea și editarea informațiilor despre o plantă:

- **ID** – identificatorul unic al plantei (generat automat)
- **Denumire** – numele plantei (ex: Venus Flytrap, Rose, Water Lily)
- **Tip** – categoria plantei (ex: Flowering, Flowers, Aquatic Plant)
- **Specie** – specia botanică (ex: Dionaea muscipula, Rosa, Nymphaea)
- **Carnivoră** – checkbox pentru a marca dacă planta este carnivoră
- **Imagine** – calea către imaginea plantei

Butoanele de acțiune

- **Adaugă** – inserează o plantă nouă în baza de date
- **Editează** – modifică informațiile plantei selectate
- **Șterge** – elimină planta selectată din baza de date
- **Afișează toate** – reîncarcă lista completă de plante

Secțiunea „Lista plantelor" Afișează toate plantele într-un tabel cu coloanele: ID, Denumire, Tip, Specie, Carnivoră și Imagine. Fiecare rând include o miniatură a imaginii plantei pentru identificare vizuală rapidă.

Secțiunea „Filtrare & Sortare" Permite:

- Filtrarea plantelor după: Tip, Specie sau status Carnivoră
- Sortarea listei după diferite criterii (denumire, tip, specie)

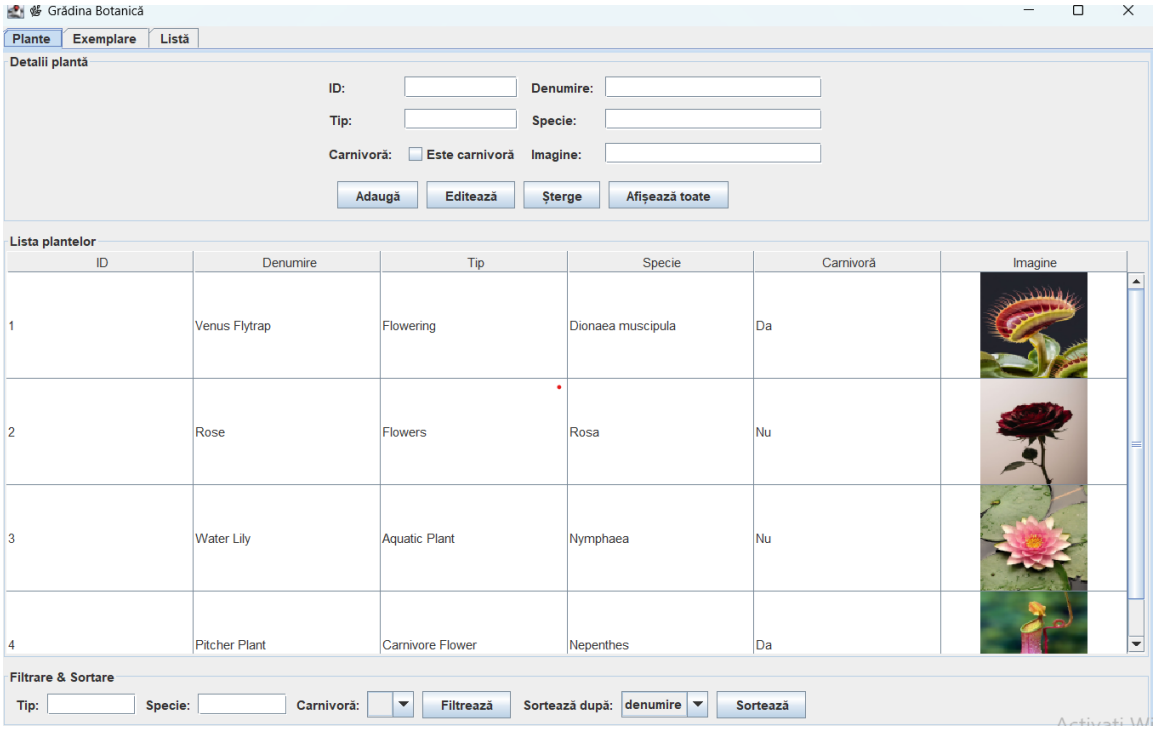

Figura 6: Interfața grafică – Tab-ul Plante

5.2 Tab-ul Exemplare

Acest tab permite gestionarea exemplarelor individuale ale speciilor din grădina botanică, cu funcționalități similare pentru adăugare, editare, ștergere și căutare.

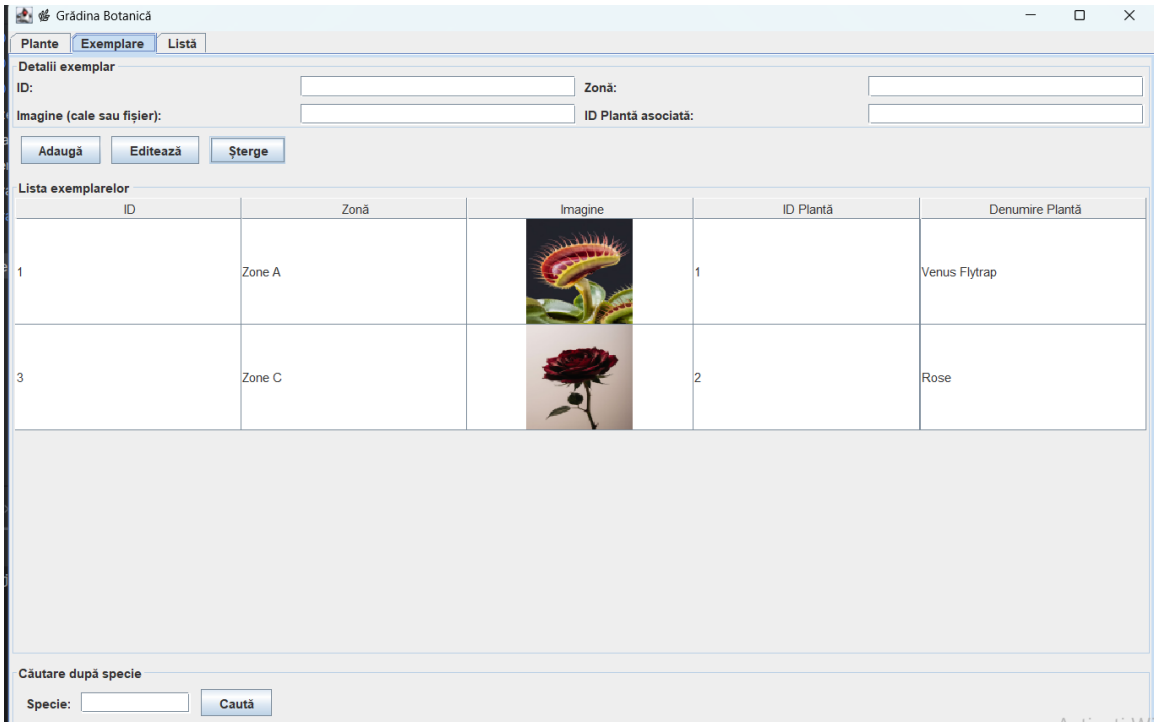

Figura 7: Interfața grafică – Tab-ul Exemplare

5.3 Tab-ul Listă

Acest tab oferă o vizualizare de ansamblu a tuturor plantelor și exemplarelor, cu opțiuni avansate de filtrare și sortare.

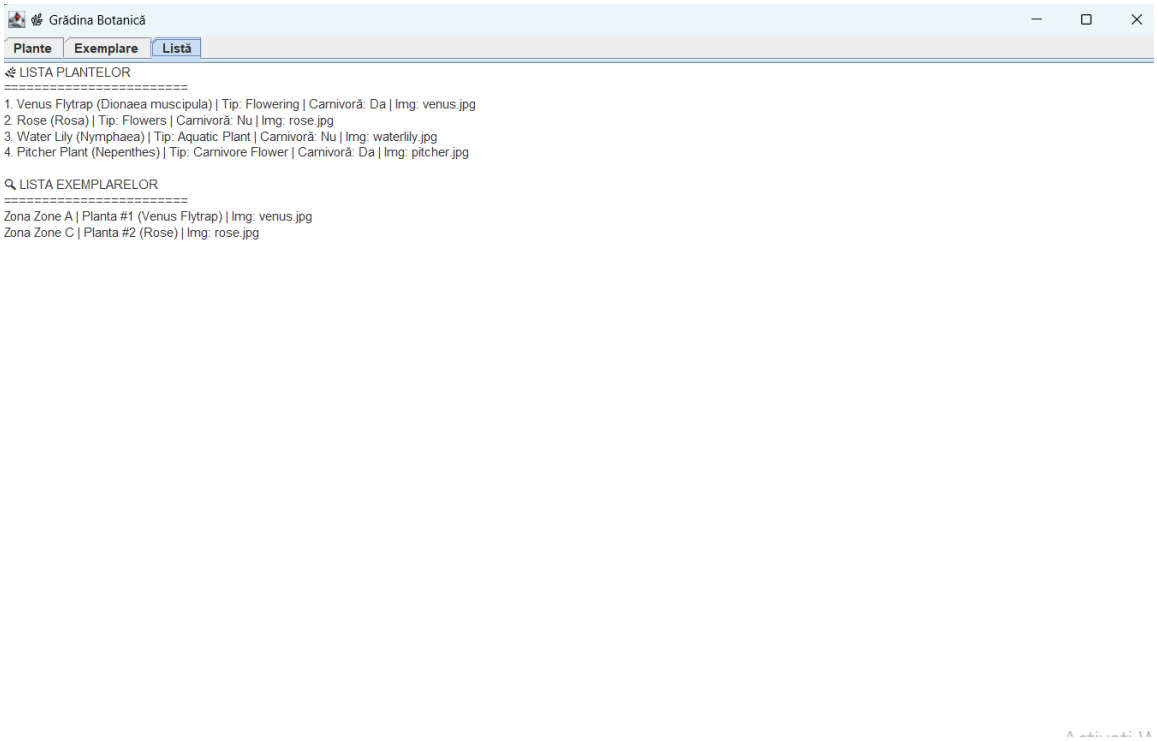

Figura 8: Interfața grafică – Tab-ul Listă

6 Concluzii

Aplicația dezvoltată oferă o soluție completă și funcțională pentru gestionarea plantelor într-o grădină botanică, îndeplinind toate cerințele specificate în enunț.

6.1 Obiective realizate

- **Gestionarea plantelor** – implementarea completă a operațiilor CRUD (Create, Read, Update, Delete) pentru entitatea Planta, incluzând suport pentru imagini și clasificarea plantelor carnivore.
- **Gestionarea exemplarelor** – posibilitatea de a evidenția exemplarele individuale ale fiecărei specii, cu informații despre localizarea acestora în grădină.
- **Vizualizare și filtrare** – interfață intuitivă pentru listarea, filtrarea și sortarea datelor după multiple criterii (tip, specie, carnivoră).
- **Căutare avansată** – funcționalitate de căutare a exemplarelor dintr-o anumită specie.

6.2 Aspecte tehnice

Arhitectura **Model-View-Presenter (MVP)** adoptată asigură:

- **Separarea responsabilităților** – fiecare componentă are un rol clar definit, ceea ce facilitează înțelegerea și modificarea codului.
- **Testabilitate** – logica de business din Presenter poate fi testată independent de interfața grafică.
- **Mentenabilitate** – modificările într-un strat nu afectează celelalte straturi, atâta timp cât interfețele rămân consistente.
- **Reutilizabilitate** – componentele Model și Repository pot fi refolosite în alte contexte (ex: aplicație web).

6.3 Posibile îmbunătățiri

Pentru dezvoltări viitoare, aplicația ar putea fi extinsă cu:

- Sistem de autentificare pentru diferite tipuri de utilizatori (administrator, vizitator)
- Export de rapoarte în format PDF sau Excel
- Integrare cu un sistem de notificări pentru îngrijirea plantelor
- Hartă interactivă a grădinii botanice cu localizarea exemplarelor
- Versiune web sau mobilă a aplicației

6.4 Experiență dobândită

Dezvoltarea acestui proiect a permis consolidarea cunoștințelor în:

- Programare orientată pe obiecte în Java
- Proiectarea și implementarea bazelor de date relaționale
- Utilizarea pattern-urilor arhitecturale (MVP)
- Dezvoltarea interfețelor grafice cu Java Swing
- Modelarea sistemelor software folosind diagrame UML